

Agentic AI for Code Generation

Seminar Report

Student: "Thua Duc Nguyen"✉ and **Supervisor:** "Derui Zhu"✉

TUM School of Computation, Information and Technology, Technical University of Munich

✉ thuaduc.nguyen@tum.de

✉ derui.zhu@tum.de

April 1, 2025

Abstract —

1 Introduction

Motivation. The rise of “vibe coding” and AI-assisted software development: how natural-language-driven programming is reshaping the developer workflow.

Background and Related Surveys. Positioning relative to prior surveys on LLM-based code generation, automated program repair, and AI agent architectures. What this report adds: a pipeline-centric synthesis across four active research fronts.

Scope and Contribution. Define the scope (autonomous code-generation agents, 2023–2025 literature), state the contribution (unified pipeline framing, cross-topic synthesis), and outline the report structure.

2 The Vibe Coding Pipeline

Present the vibe coding pipeline as a unifying conceptual framework and compare it to classical SE process models (waterfall, agile inner loops).

2.1 Pipeline Stages

Formal definition of each stage: Intent Capture → Planning → Code Generation → Execution → Refinement. Five stages forming a loop: execution feedback drives refinement, which may trigger re-planning or re-generation. What inputs and outputs connect adjacent stages.

2.2 Mapping to Survey Topics

Explicit mapping: how each of the four subsequent sections corresponds to one or more pipeline stages. §3 covers the full pipeline as an integrated system;

§4 zooms into planning; §5 covers multi-agent approaches to generation and validation; §6 covers the execution-feedback-refinement loop (including fault-specific self-repair). Cross-cutting concerns (evaluation, context management) are consolidated in the Discussion.

3 End-to-End Autonomous Software Engineering Agents

The big picture first: systems that orchestrate the entire pipeline—from a natural-language issue to a tested pull request—showing how the stages compose into a holistic system before we zoom into each stage.

3.1 Architecture and Orchestration

Common architectural choices: tool-augmented LLMs, ReAct-style loops, memory modules, context management strategies, and how agents interface with codebases and dev tools. Emphasis on the orchestration layer that ties pipeline stages together.

3.2 Representative Systems

Concrete systems (e.g., SWE-Agent, Devin, OpenHands, AutoCodeRover, Agentless) with brief descriptions of their design and reported full-pipeline performance.

3.3 Full-Pipeline Evaluation

How end-to-end agents are evaluated: SWE-bench resolution rates, real-world issue benchmarks, and the methodology behind current leaderboards. Critical analysis of benchmark limitations is deferred to the Discussion (§7.2).

4 Planning and Search Strategies for Code Synthesis

Zooming into the first critical stage: how agents decide *what* to generate before or during generation—decomposing problems, exploring solution spaces, and selecting among candidates.

4.1 Plan Representations and Decomposition

Pseudocode plans, hierarchical task decomposition, chain-of-thought outlines, dependency graphs, and specification sketches used to guide generation.

4.2 Search and Exploration Strategies

Tree/graph search over partial programs (MCTS, beam search, best-first), sampling-and-ranking, evolutionary approaches, and backtracking mechanisms.

4.3 Plan–Generate Interplay

How planning interleaves with code generation: plan-then-generate vs. interleaved planning, plan revision upon execution feedback, and plan grounding.

5 Multi-Agent Frameworks for Code Generation

An alternative architectural approach to the generation-to-validation span: multiple specialized agents divide labor by role (architect, coder, tester, reviewer), collectively covering planning, generation, and internal verification to improve code quality beyond what a single agent achieves.

5.1 Role Decomposition and Communication

How tasks are split among agents, what roles exist, and how agents exchange information (shared blackboard, message passing, structured dialogue).

5.2 Framework Designs

Survey of concrete frameworks (e.g., ChatDev, MetaGPT, AutoGen, CrewAI) and their orchestration mechanisms.

5.3 When Does Multi-Agency Help?

Conditions under which multi-agent designs outperform monolithic agents; coordination overhead, failure modes, and diminishing returns with agent count.

6 Execution Feedback and Iterative Refinement

Closing the loop: agents execute generated code, observe runtime outcomes (test failures, compiler errors, exceptions), and revise their output. Covers both general refinement and fault-specific self-repair.

6.1 Feedback Signals and Environments

Types of feedback: test results, compiler errors, runtime exceptions, static analysis, human approval signals. Sandboxed execution environments and their design.

6.2 Iterative Refinement Mechanisms

How agents incorporate feedback: reflexion-style verbal self-critique, repair prompts, diff-based editing, and multi-turn conversation with the environment.

6.3 Fault Localization and Self-Repair

When refinement becomes debugging: stack-trace analysis, LLM-driven root-cause reasoning, edit-based patching, candidate ranking, and test-driven repair loops.

6.4 Convergence and Stopping Criteria

When to stop iterating: pass-rate thresholds, diminishing improvement detection, budget limits, and the risk of over-fitting to weak test suites.

7 Discussion

7.1 Consolidated Patterns and Trends

Recurring design patterns across all five topics (observe-think-act loops, tool orchestration, LLM-as-judge verification). Macro trends: from single-shot prompting to agentic scaffolds, from monolithic agents to composable pipelines, from flat CoT to tree-structured search.

7.2 The Evaluation Landscape

Cross-cutting view of benchmarks (HumanEval, MBPP, SWE-bench, Defects4J, CrossCodeEval), their coverage of pipeline stages, saturation concerns, and gaps in realism.

7.3 The Context Window Bottleneck

How finite context constrains every pipeline stage; retrieval-augmented generation, hierarchical summarization, and repository-level chunking as mitigations.

7.4 Open Challenges

Unsolved problems: long-horizon reasoning, specification ambiguity, cost efficiency, trustworthiness, and human-in-the-loop control design.

7.5 Implications for Software Engineering Practice

How these advances may reshape developer workflows, team structures, and the economics of software production.

8 Conclusion

Summarize key findings across the five research fronts, restate the pipeline framing as a unifying lens, and point to the most promising future directions.